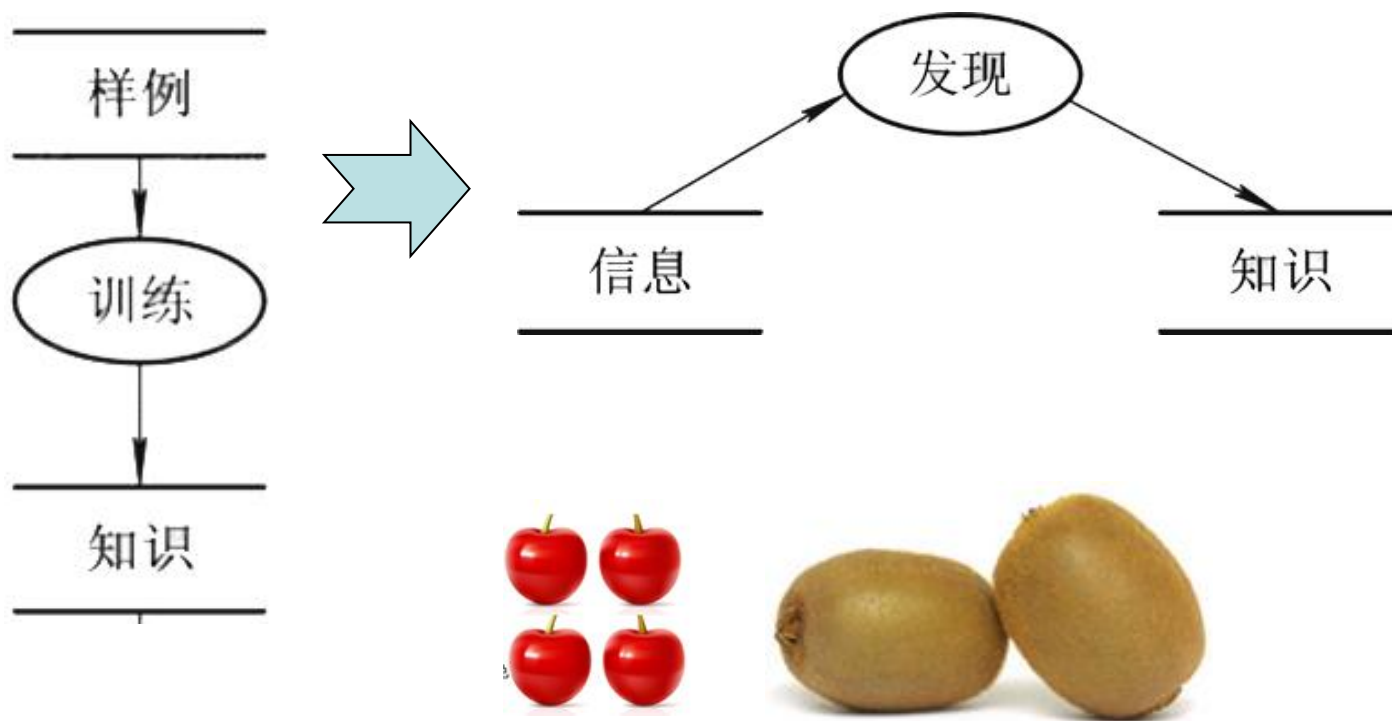


机器学习

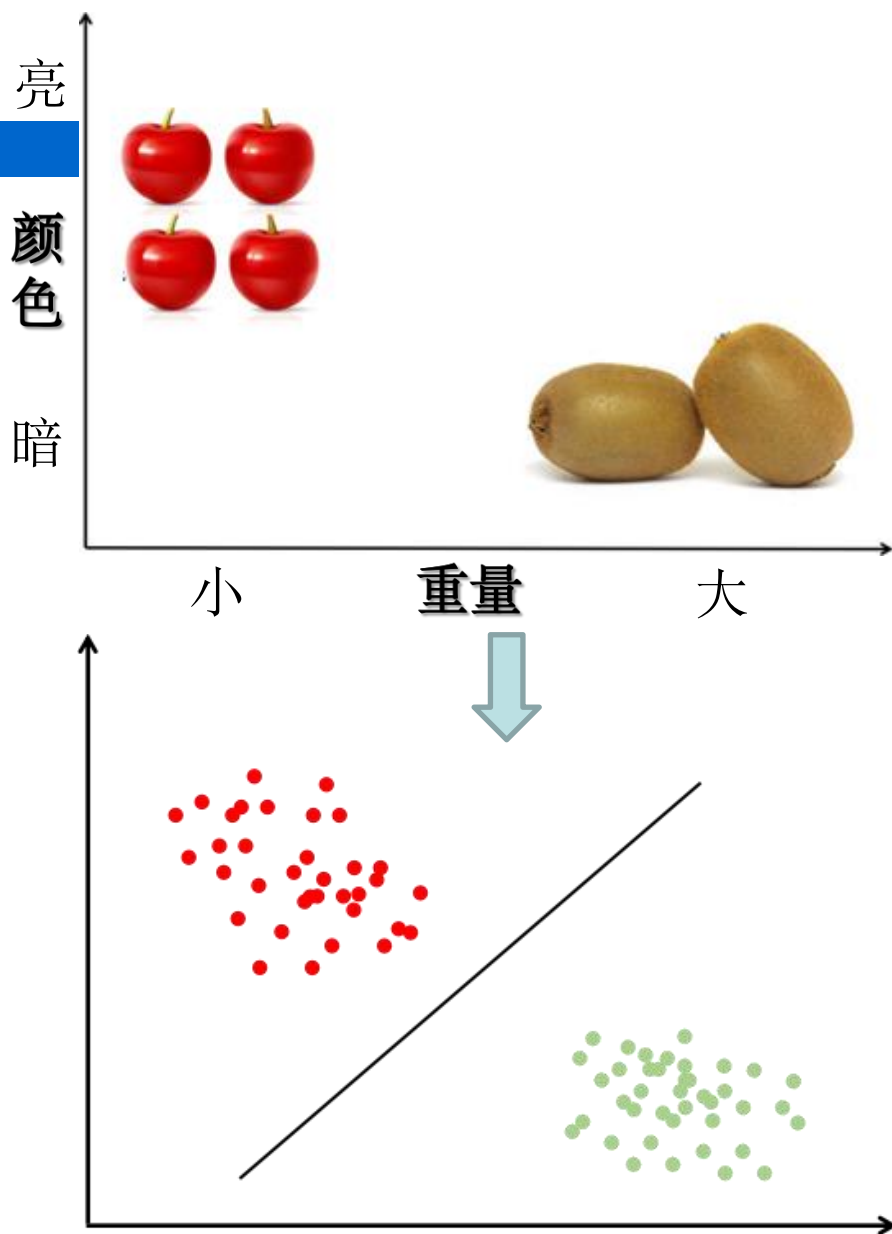
机器学习的核心思想



机器学习

樱桃的**颜色亮**，**重量小**；猕猴桃的**颜色暗**，**重量大**；

收集一些樱桃、猕猴桃，作为**训练样本**，测量它们的这两个特征；樱桃位于xy平面的左上角，猕猴桃位于右下角。



机器学习

- 用一条直线可以将这两种类型分开，这条直线称为**线性分类器**。

- 直线方程为 $ax + by + c = 0$

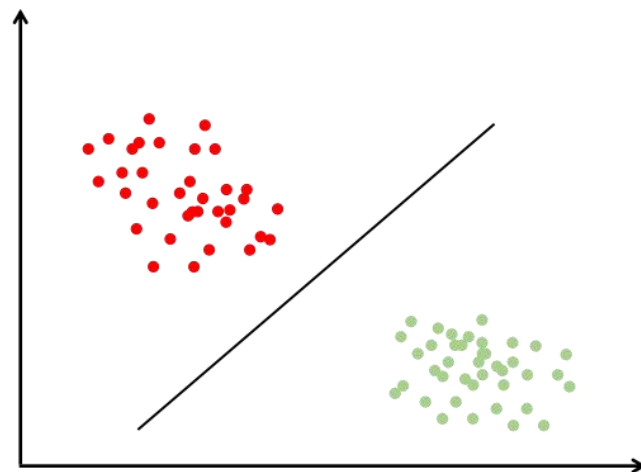
- 落在直线上方的点被判定成樱桃

$$ax + by + c < 0$$

- 落在直线下方的点被判定成猕猴桃

$$ax + by + c > 0$$

- 这条直线就是训练得到的机器学习模型，直线方程的参数就是机器学习模型参数

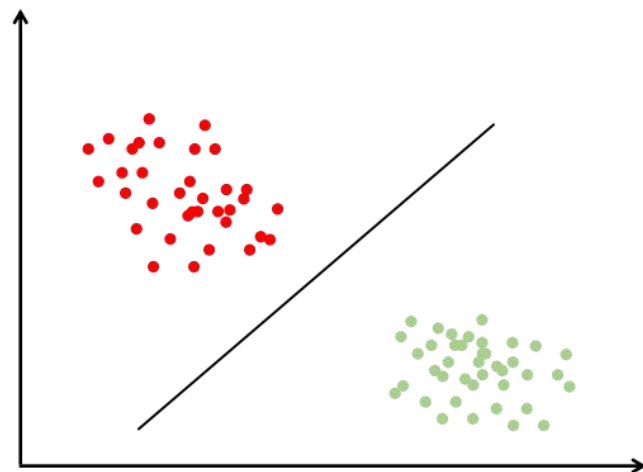


机器学习

正负样本：给两类水果编号，称为类别标签，在这里樱桃类别标签为-1，猕猴桃类别标签为+1

判定规则可以写成**决策函数**：

$$f(x, y) = \begin{cases} +1 & ax + by + c > 0 \\ -1 & ax + by + c \leq 0 \end{cases}$$

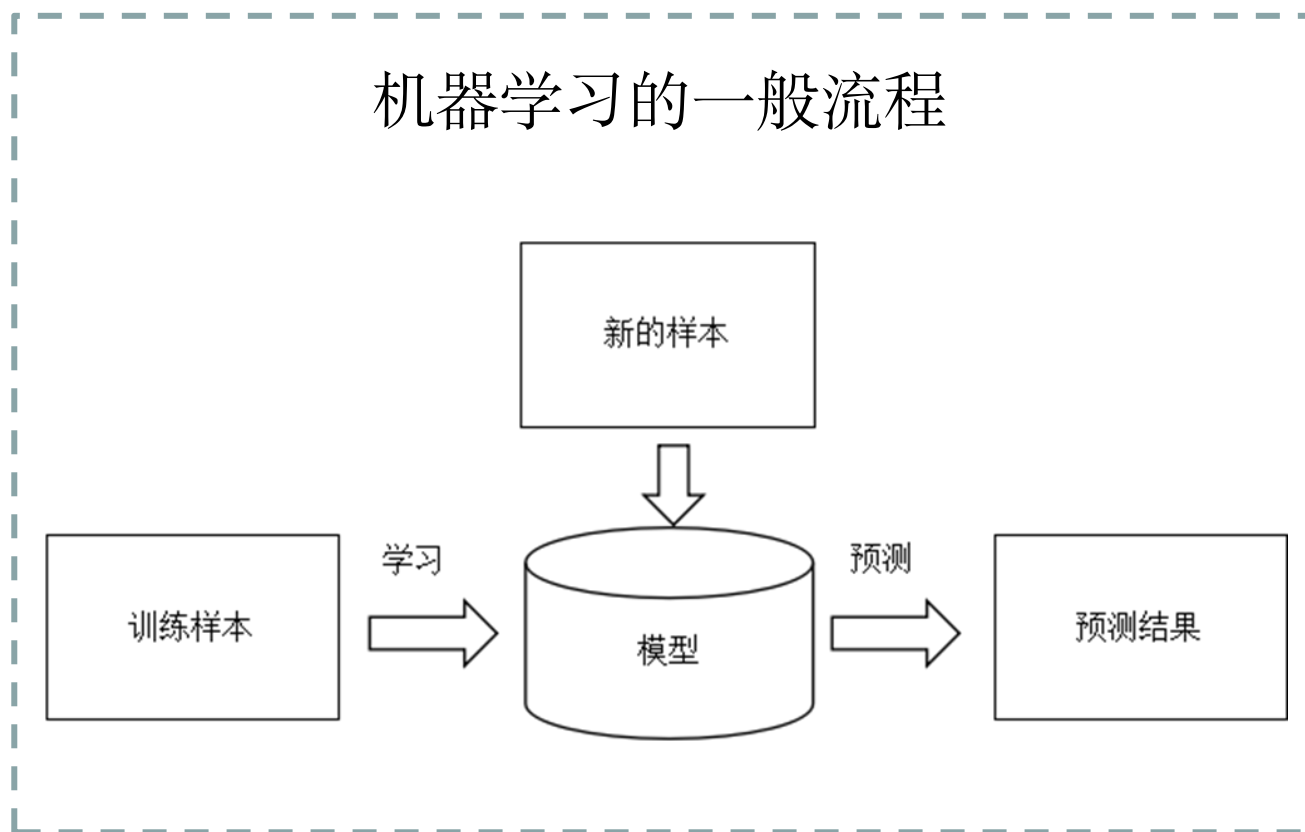


问题：如何找到这条直线？

通过训练样本训练得到，保证对样本大部分**都能**正确的分类。

机器学习

机器学习的一般流程



训练集和测试集

- **训练集:** 用于训练, 构建模型
- **测试集:** 在模型检验时使用, 用于评估模型是否有效

关键: 怎么获得“测试集” (test set) ?

测试集应该与训练集“互斥”

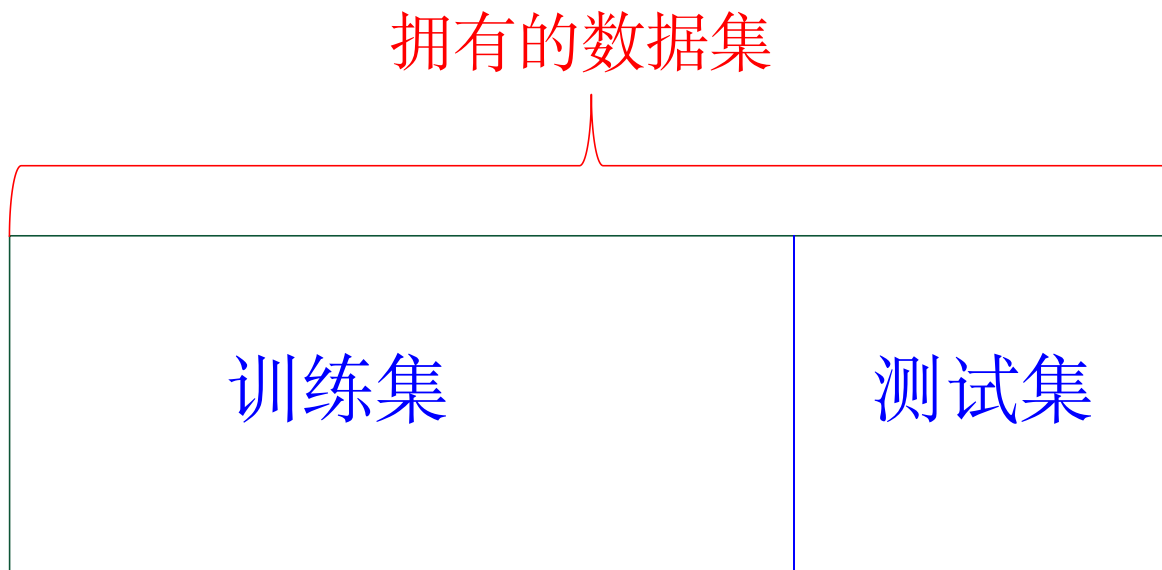
常见方法:

留出法 (hold-out)

交叉验证法 (cross validation)

自助法 (bootstrap)

留出法



注意:

保持数据分布一致性 (例如: 分层采样)

多次重复划分 (例如: 100次随机划分)

测试集不能太大、不能太小 (例如: $1/5 \sim 1/3$)

K-折交叉验证法

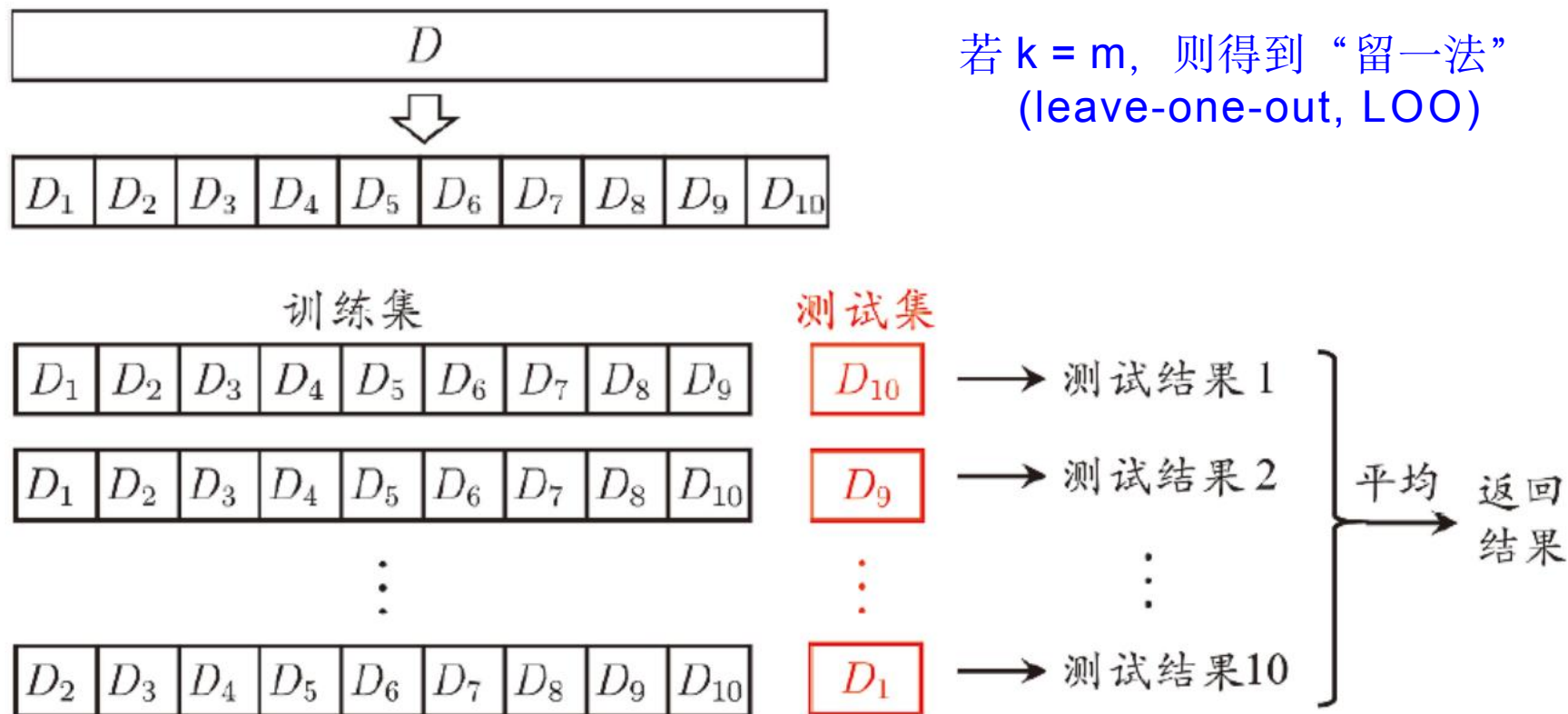
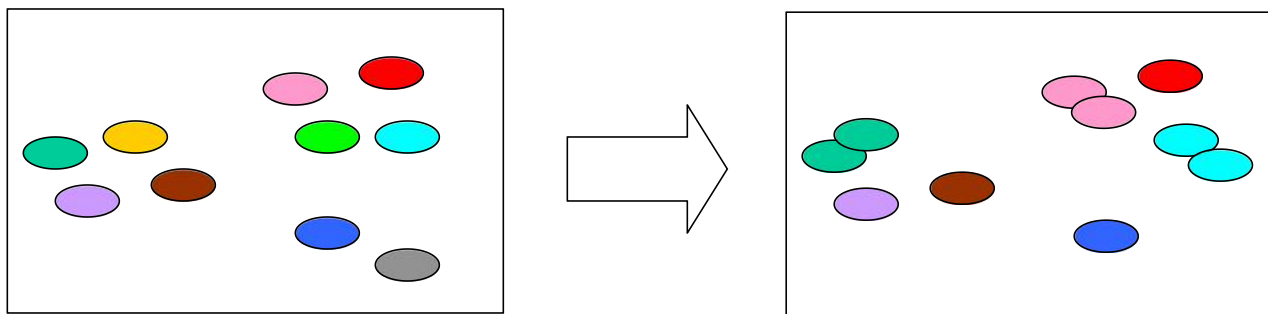


图 2.2 10 折交叉验证示意图

自助法

基于“自助采样” (bootstrap sampling)

亦称“有放回采样”、“可重复采样”



训练集与原样本集同规模

数据分布有所改变

自助法

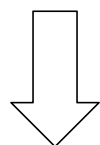
假设我们有一个包含10个样本的数据集:

$$D = \{A, B, C, D, E, F, G, H, I, J\}$$

使用自助法, 我们从这个数据集中随机抽取10个样本, 允许重复。可能的抽样结果如下:

$$\{A, A, C, D, E, E, F, G, I, J\}$$

约有 36.8% 的样本不出现


$$\lim_{m \rightarrow \infty} \left(1 - \frac{1}{m}\right)^m \mapsto \frac{1}{e} \approx 0.368$$

“包外估计” (out-of-bag estimation)

过拟合与欠拟合

- **训练集**——课本，学生根据课本里的内容来掌握知识
- **测试集**——考试，考的题是平常都没有见过，考察学生举一反三的能力
- 机器学习的主要挑战是算法必须能够在**先前未观测的新输入**上表现良好，**而不只是在训练集上表现良好**。这种在先前未观测到的输入上表现良好的能力被称为 **泛化**。

好的机器学习模型：



泛化能力强!

过拟合与欠拟合

泛化误差 vs. 经验误差

泛化误差：在“未来”样本上的误差

经验误差：在训练集上的误差，亦称“训练误差”

泛化误差越小越好

经验误差是否越小越好？

NO! 因为会出现“**过拟合**” (overfitting)

过拟合与欠拟合

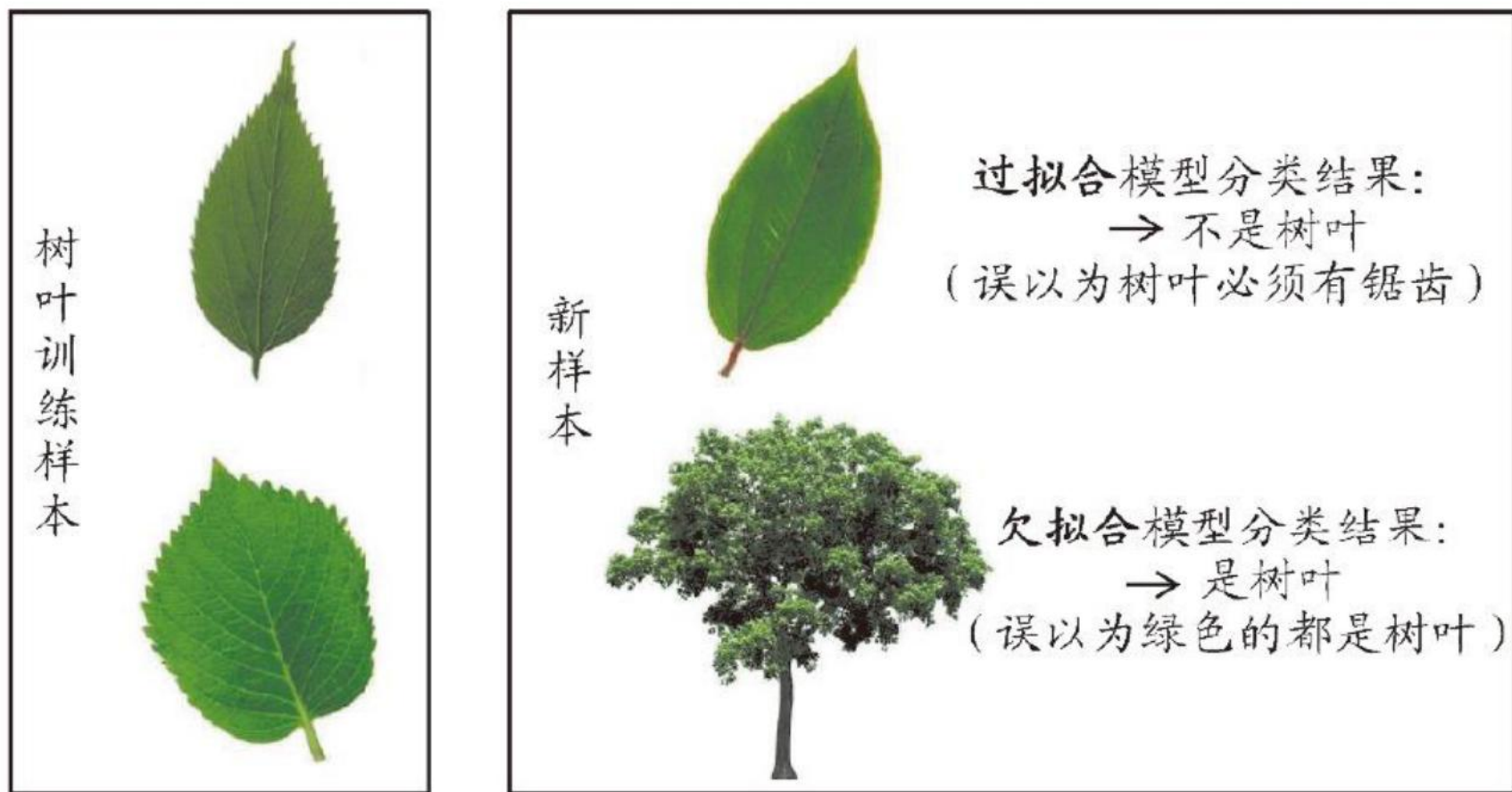


图 2.1 过拟合、欠拟合的直观类比

过拟合与欠拟合

- 决定机器学习算法效果好坏的因素:

1. 训练误差高、测试误差高



欠拟合

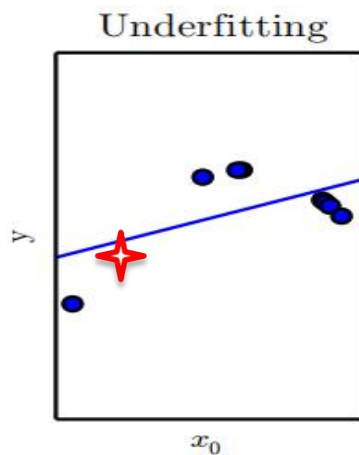
2. 训练误差低、测试误差误差高



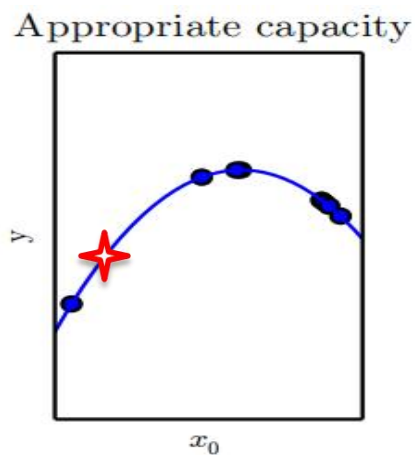
过拟合

- 欠拟合是指模型不能在训练集上获得足够低的误差;
- 过拟合是指训练误差虽然低, 但是测试误差很高。

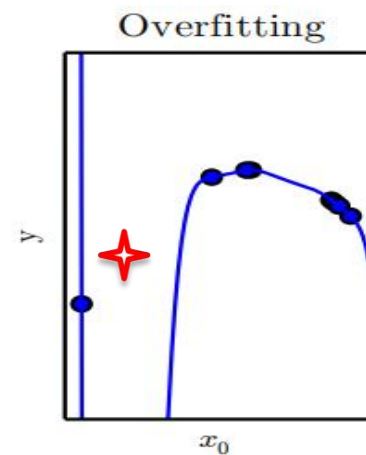
过拟合与欠拟合



$$\hat{y} = b + wx.$$

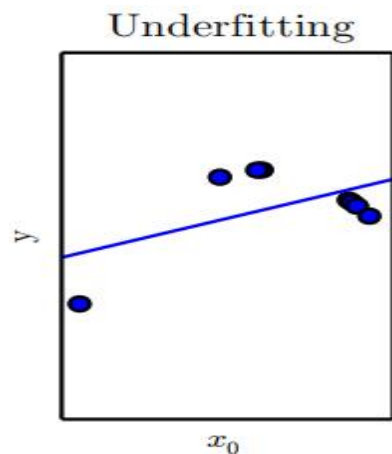


$$\hat{y} = b + w_1x + w_2x^2.$$

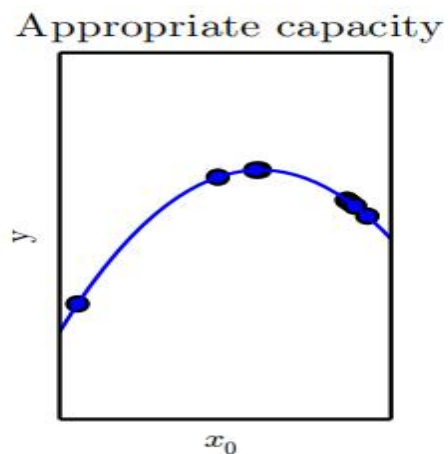


$$\hat{y} = b + \sum_{i=1}^9 w_i x^i.$$

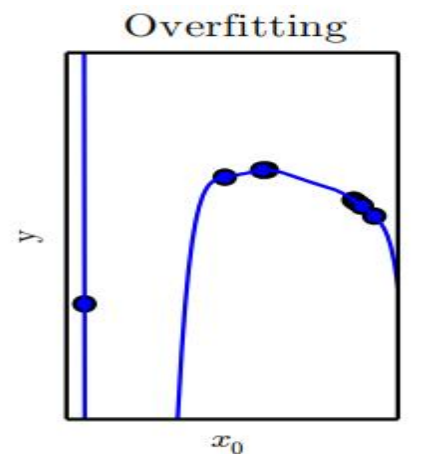
模型选择



$$\hat{y} = b + wx.$$



$$\hat{y} = b + w_1x + w_2x^2.$$



$$\hat{y} = b + \sum_{i=1}^9 w_i x^i.$$

超参数是人为设置的，不是通过学习算法本身学习出来的

模型选择

算法的参数：一般由人工设定，亦称“超参数”

模型的参数：一般由学习确定

参数调得好不好对性能往往对最终性能有关键影响

那我们怎么获得无法学习到的“好”的超参数呢？

模型选择

- 训练集——课本，学生根据课本里的内容来掌握知识
- 测试集——考试，考的题是平常都没有见过，考察学生举一反三的能力

验证集——作业，评估学习情况、进度快慢



选超参

模型选择

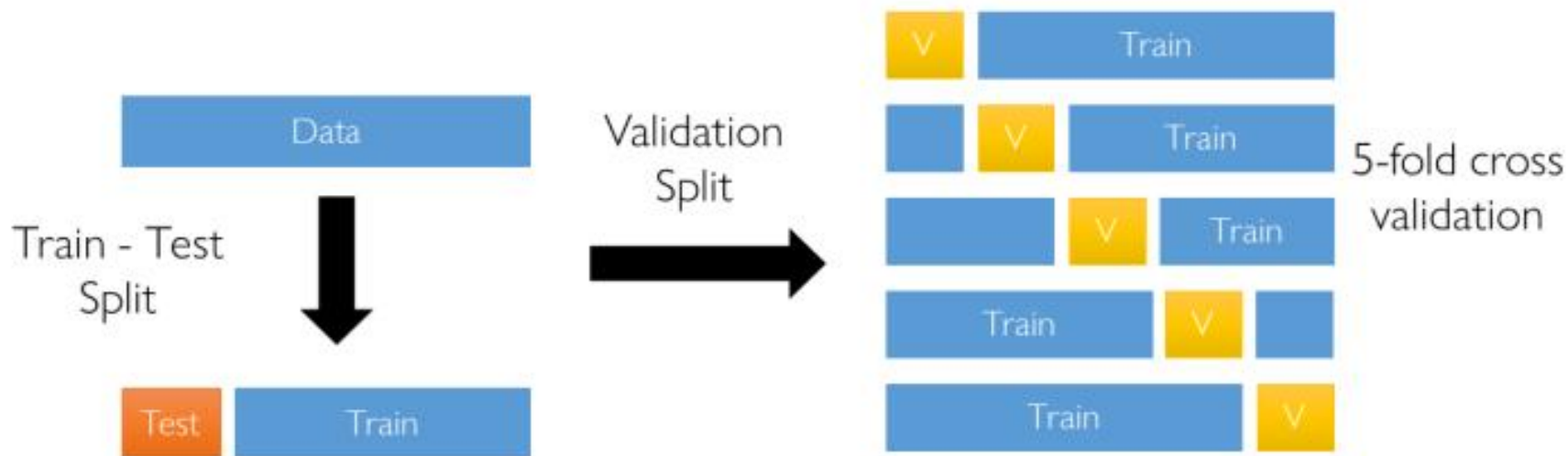
训练集 vs. 测试集 vs. 验证集 (validation set)

训练集：用于训练模型，优化**模型的参数**。模型通过该数据集学习特征与标签之间的关系。

验证集：用于**调优超参数**和评估模型性能。验证集的主要作用是帮助选择最佳的超参数组合。

测试集：用于最终评估模型的**泛化能力**。在超参数调优完成后，使用测试集来评估模型的最终性能，以确定其在新数据上的表现。

交叉验证法选超参数



第一步，先把测试集拿出去

平均交叉验证误差

性能度量

什么样的模型是“好”的，不仅取决于算法和数据，
还取决于任务需求

性能度量(performance measure)是衡量模型泛化能力的评价标准，反映了任务需求，使用不同的性能度量往往会导致不同的评判结果

回归(regression) 任务常用均方误差：

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m (f(x_i) - y_i)^2$$

性能度量

错误率:

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) \neq y_i)$$

精度:

$$\begin{aligned} \text{acc}(f; D) &= \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) = y_i) \\ &= 1 - E(f; D) . \end{aligned}$$

性能度量

表 2.1 分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

TN, 预测是负样本, 预测对了

FP, 预测是正样本, 预测错了

FN, 预测是负样本, 预测错了

TP, 预测是正样本, 预测对了

- 模型预测结果: 0, 1, 0, 0, 1
- 实际样本结果: 1, 1, 0, 1, 0

性能度量

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

$$\text{准确率} = \frac{\text{所有预测正确的样本}}{\text{总样本}} = \frac{TP + TN}{TP + FN + FP + TN}$$

$$\text{精确率} = \frac{\text{将正类预测为正类}}{\text{预测的正类}} = \frac{TP}{TP + FP}$$

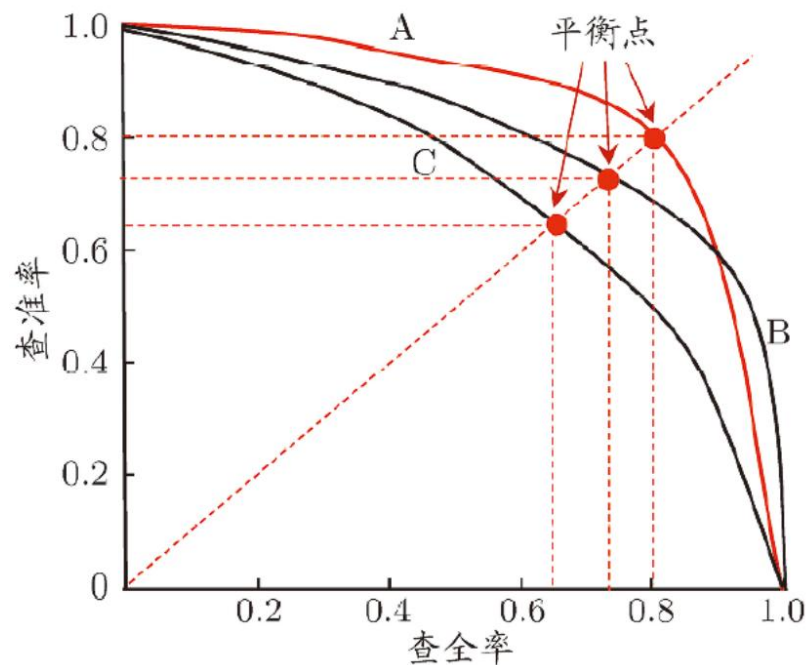
$$\text{召回率} = \frac{\text{将正类预测为正类}}{\text{原本的正类}} = \frac{TP}{TP + FN}$$

- **准确率**: 分类正确的样本数占样本总数的比例, 通过记录正确的数量占比即可。
- **精确率、查准率**: 是针对预测结果而言的, 它的含义是在所有被预测为正的样本中实际为正的样本的概率
- **召回率、查全率**: 是针对原样本而言的, 它的含义是在实际为正的样本中被预测为正样本的概率
- 模型预测结果: 0, 1, 0, 0, 1
- 实际样本结果: 1, 1, 0, 1, 0

性能度量

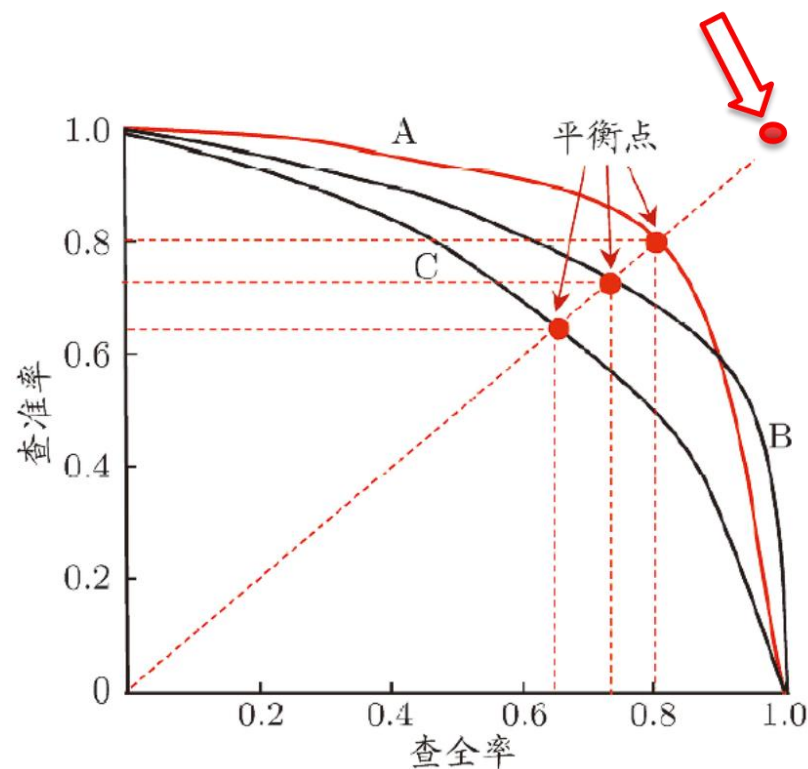
计算PR图

- 排序：将预测结果概率值排序
- 降低阈值：将概率阈值由1开始逐渐降低，按此顺序逐个预测样本
- 计算精确率P和召回率R：在每个阈值下，计算出P和R。
- 绘制曲线：以P为纵坐标，R为横坐标绘制点



性能度量

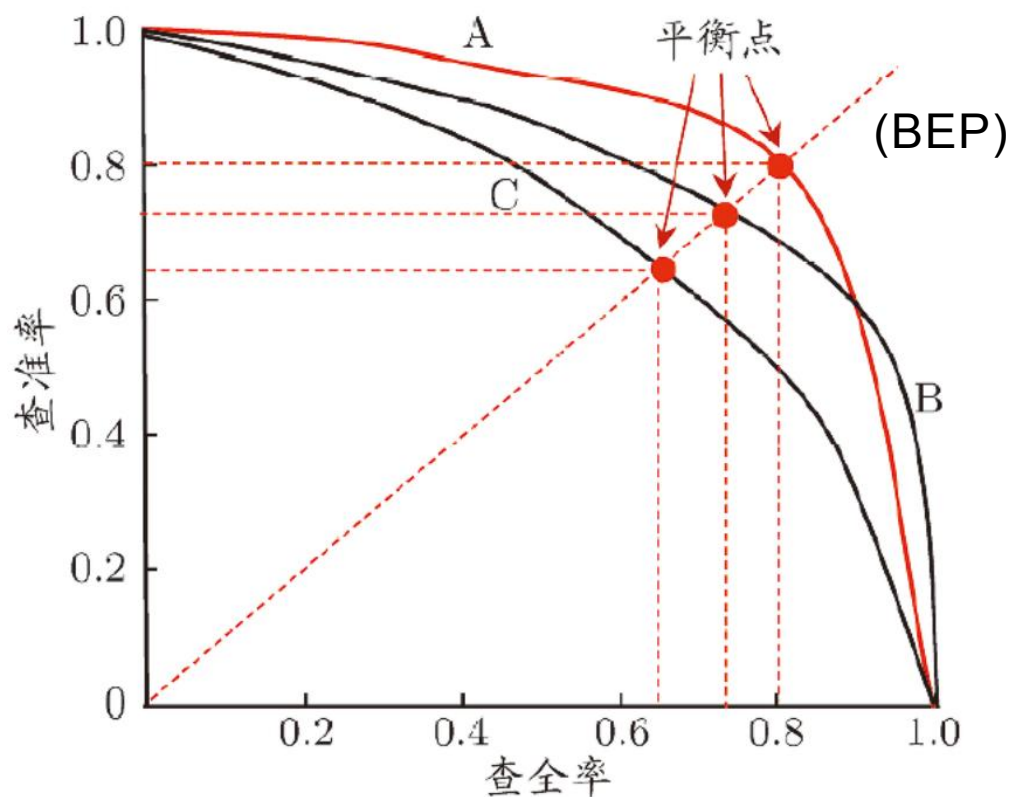
- 理想情况下，我们希望模型能在高召回率的同时保持高精确率
- 右上角代表理想的模型性能。在这个点上，召回率为1（所有正例都被识别）且精确率也为1（所有预测的正例都是正确的）。因此，曲线越靠近右上角，表示模型能够以高召回率捕获更多的正例，同时保持较低的假正例率，说明模型的整体性能更佳。



性能度量

PR图, BEP

BEP: 精确率与召回率达到最佳平衡的点, 即精确率等于召回率时的值



PR图:

- 学习器 A 优于 学习器 C
- 学习器 B 优于 学习器 C
- 学习器 A ?? 学习器 B

BEP:

- 学习器 A 优于 学习器 B
- 学习器 A 优于 学习器 C
- 学习器 B 优于 学习器 C

性能度量

F1 Score: 是精确率和召回率的调和平均:

$$H_n = \frac{1}{\frac{1}{n} \sum_{i=1}^n \frac{1}{x_i}} = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$$

$$F1 = \frac{2 \times P \times R}{P + R} = \frac{2 \times TP}{\text{样例总数} + TP - TN}$$

若对查准率/查全率有不同偏好:

$$F_\beta = \frac{(1 + \beta^2) \times P \times R}{(\beta^2 \times P) + R}$$

$\beta > 1$ 时查全率有更大影响; $\beta < 1$ 时查准率有更大影响

性能度量

我们有三次实验的混淆矩阵如下：

实验 1

	预测正类	预测负类
实际正类	5	2
实际负类	1	4

实验 2

	预测正类	预测负类
实际正类	6	1
实际负类	2	3

实验 3

	预测正类	预测负类
实际正类	4	3
实际负类	1	5

性能度量

宏XX vs. 微XX

若能得到多个混淆矩阵:

(例如多次训练/测试的结果, 多分类的两两混淆矩阵)

宏(macro-)查准率、查全率、F1

计算每个类别的F1得分, 然后取平均

$$\text{macro-}P = \frac{1}{n} \sum_{i=1}^n P_i ,$$

$$\text{macro-}R = \frac{1}{n} \sum_{i=1}^n R_i ,$$

$$\text{macro-}F1 = \frac{2 \times \text{macro-}P \times \text{macro-}R}{\text{macro-}P + \text{macro-}R} .$$

微(micro-)查准率、查全率、F1

计算总体TP、FP、FN后再计算精确率和召回率

$$\text{micro-}P = \frac{\overline{TP}}{\overline{TP} + \overline{FP}} ,$$

$$\text{micro-}R = \frac{\overline{TP}}{\overline{TP} + \overline{FN}} ,$$

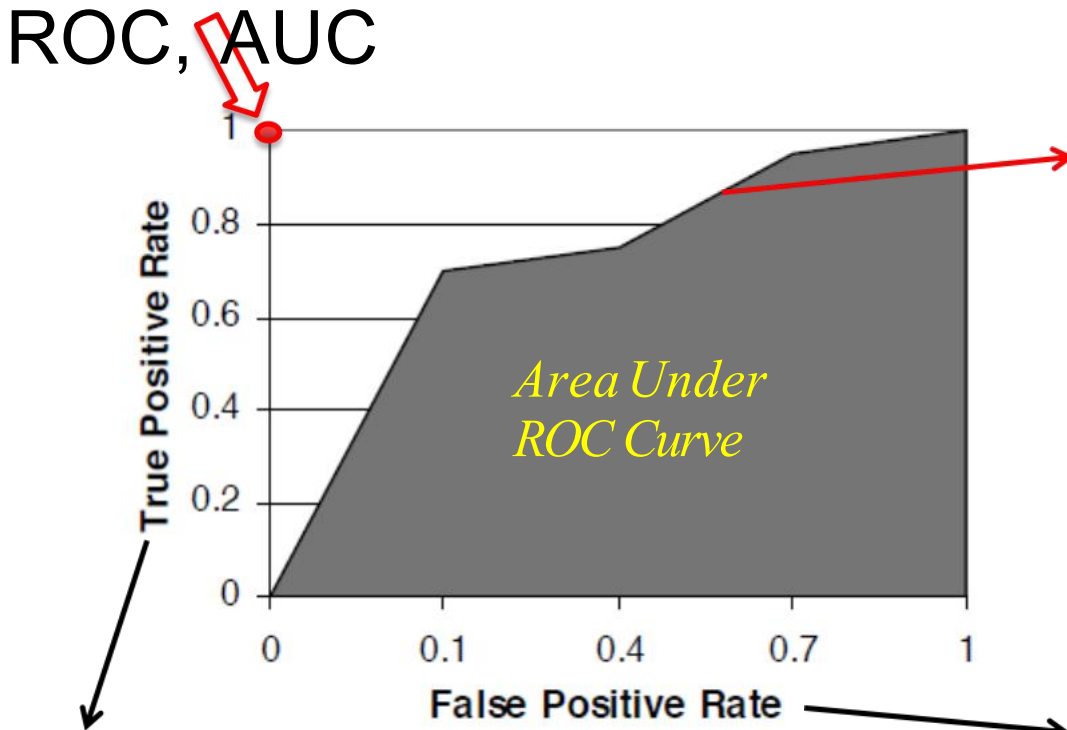
$$\text{micro-}F1 = \frac{2 \times \text{micro-}P \times \text{micro-}R}{\text{micro-}P + \text{micro-}R} .$$

性能度量

指标	计算方式	适用场景
精确率	$\frac{TP}{TP+FP}$	关注预测准确性时
召回率	$\frac{TP}{TP+FN}$	关注覆盖率时
F1 Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	综合评估性能
Macro F1	各类别F1得分平均	类别不平衡时
Micro F1	总体TP、FP、FN计算得分	整体性能评估

性能度量

ROC, AUC



ROC (Receiver Operating Characteristic) Curve [Green & Swets, Book 66; Spackman, IWML'89]

The bigger, the better

$$tpr = \frac{TP}{TP + FN} = \frac{TP}{m_+}$$

$$fpr = \frac{FP}{FP + TN} = \frac{FP}{m_-}$$

$$AUC = 1 - \frac{1}{m^+ m^-} \sum_{x^+ \in D^+} \sum_{x^- \in D^-} \left(\mathbb{I}(f(x^+) < f(x^-)) + \frac{1}{2} \mathbb{I}(f(x^+) = f(x^-)) \right)$$

性能度量

非均等代价

犯不同的错误往往会造成不同的损失

此时需考虑“非均等代价”
(unequal cost)

代价敏感(cost-sensitive)错误率:

$$E(f; D; cost) = \frac{1}{m} \left(\sum_{\mathbf{x}_i \in D^+} \mathbb{I}(f(\mathbf{x}_i) \neq y_i) \times cost_{01} + \sum_{\mathbf{x}_i \in D^-} \mathbb{I}(f(\mathbf{x}_i) \neq y_i) \times cost_{10} \right)$$

表 2.2 二分类代价矩阵

真实类别	预测类别	
	第 0 类	第 1 类
第 0 类	0	$cost_{01}$
第 1 类	$cost_{10}$	0

偏差-方差分解

对回归任务，泛化误差可通过“偏差-方差分解”拆解为：

$$E(f; D) = \underbrace{bias^2(\mathbf{x})}_{\text{偏差}} + \underbrace{var(\mathbf{x})}_{\text{方差}} + \underbrace{\varepsilon^2}_{\text{噪声}}$$

期望输出与真实
输出的差别

$$bias^2(\mathbf{x}) = (\bar{f}(\mathbf{x}) - y)^2$$

同样大小的训练集
的变动，所导致的
性能变化

$$var(\mathbf{x}) = \mathbb{E}_D \left[(f(\mathbf{x}; D) - \bar{f}(\mathbf{x}))^2 \right]$$

表达了当前任务上任何学习算法
所能达到的期望泛化误差下界

$$\varepsilon^2 = \mathbb{E}_D \left[(y_D - y)^2 \right]$$

训练样本的标记与
真实标记有区别

泛化性能是由学习算法的能力、数据的充分性以及学习任务本身的难度共同决定

偏差-方差分解

一般而言，偏差与方差存在冲突：

训练不足时，学习器拟合能力不强，偏差主导

随着训练程度加深，学习器拟合能力逐渐增强，方差逐渐主导

训练充足后，学习器的拟合能力很强，方差主导

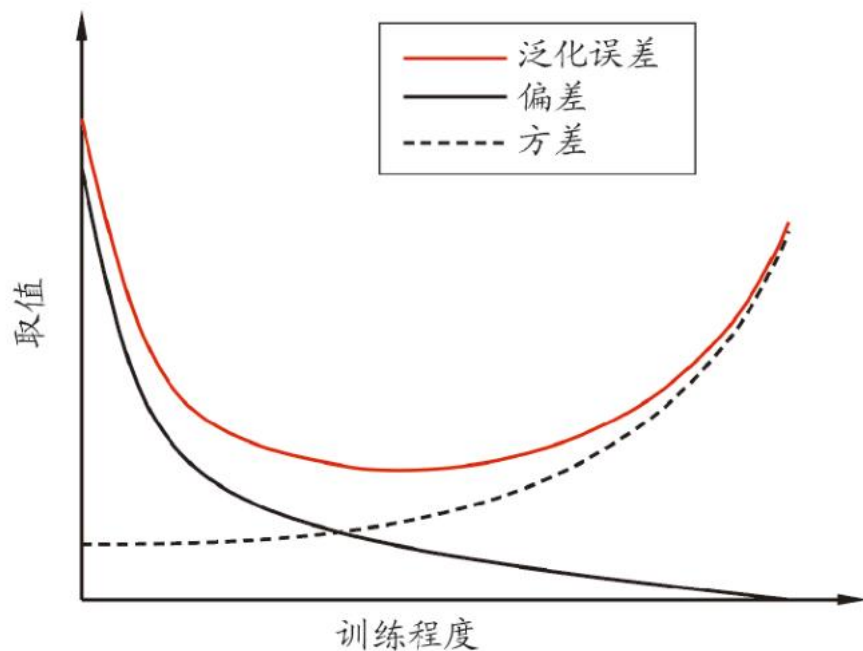


图 2.9 泛化误差与偏差、方差的关系示意图